PINNs for optimal control

Ngày 14 tháng 7 năm 2026

1 Problem formulation

Let $\Omega \subset \mathbb{R}^d$ $(d = 1, 2)$ be a bounded domain with sufficiently smooth boundary $\partial\Omega$, and let $T > 0$ be a fixed final time. We denote the space-time domain by $Q := \Omega \times (0, T)$ and the time-dependent boundary by $\Sigma := \partial\Omega \times (0, T)$.

We consider the following optimal control problem: find the state $y : Q \to \mathbb{R}$ and the boundary control $u : \Sigma \to \mathbb{R}$ that minimize the tracking-type cost functional

$$J(y, u) = \int_0^T \int_\Omega |y(x, t) - y_d(x, t)|^2 \, dx \, dt + \frac{\alpha}{2} \int_0^T \int_{\partial\Omega} |u(x, t)|^2 \, ds \, dt, \tag{1}$$

subject to the semilinear parabolic state equation

$$\begin{cases} \partial_t y - \nu \Delta y + f(y) = 0, & \text{in } Q, \\ y(x, t) = u(x, t), & \text{on } \Sigma, \\ y(x, 0) = y_0(x), & \text{in } \Omega, \end{cases} \tag{2}$$

where $\nu > 0$ is the diffusion coefficient, $y_d \in L^2(Q)$ is a given desired trajectory, and $\alpha > 0$ is fixed regularization parameter. The boundary condition in (2) is of Dirichlet type, meaning that the control acts directly on the boundary value of the state.

2 Methodology

In this section, we present two Physics-Informed Neural Network (PINN) formulations for solving the optimal control problem stated in Section 1. The first is a *direct* approach, which treats the constrained optimization problem as a penalized loss minimization. The second is an *indirect* approach, which enforces the first-order optimality system directly. Both formulations use neural networks to parameterize the unknown functions; however, the indirect formulation additionally incorporates a hard-constraint strategy to exactly impose the Dirichlet boundary condition.

2.1 Direct PINN Formulation

The direct PINN method approximates the state $y(x, t)$ and the control $u(x, t)$ by two independent neural networks:

$$y(x, t) \approx Y_\theta(x, t), \qquad u(x, t) \approx U_\psi(x, t), \tag{3}$$

where θ and ψ denote the trainable parameters of the respective networks. The networks are trained by minimizing a composite loss function that includes the PDE residual, the boundary/initial conditions, and the cost functional (as soft penalties).

Define the PDE residual for the state equation:

$$r_y(x, t; \theta, \psi) := \partial_t Y_\theta - \nu \Delta Y_\theta + f(Y_\theta). \tag{4}$$

Note that for the Dirichlet boundary control, the control u does not appear inside the PDE (unlike distributed control); it enters only through the boundary condition $Y_\theta = U_\psi$ on Σ.

The direct PINN loss is then constructed as:

$$\begin{aligned} \mathcal{L}_{\text{dir}}(\theta, \psi) = w_{\text{res}} \mathcal{L}^y_{\text{res}} + w_{\text{bc}} \mathcal{L}^y_{\text{bc}} + w_{\text{ic}} \mathcal{L}^y_{\text{ic}} \\ + \mathcal{L}_{\text{track}} + \frac{\alpha}{2} \mathcal{L}_{\text{reg}}, \end{aligned} \tag{5}$$

where the individual loss terms are defined as follows, using Monte Carlo approximations over collocation points:

$$\mathcal{L}^y_{\text{res}} := \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} |r_y(x_j, t_j; \theta, \psi)|^2 , \tag{6}$$

$$\mathcal{L}^y_{\text{bc}} := \frac{1}{N_{\text{bc}}} \sum_{j=1}^{N_{\text{bc}}} \left|Y_\theta(x^b_j, t^b_j) - U_\psi(x^b_j, t^b_j)\right|^2 , \tag{7}$$

$$\mathcal{L}^y_{\text{ic}} := \frac{1}{N_{\text{ic}}} \sum_{i=1}^{N_{\text{ic}}} |Y_\theta(x_i, 0) - y_0(x_i)|^2 , \tag{8}$$

$$\mathcal{L}_{\text{track}} := \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} |Y_\theta(x_j, t_j) - y_d(x_j, t_j)|^2 , \tag{9}$$

$$\mathcal{L}_{\text{reg}} := \frac{1}{N_{\text{bc}}} \sum_{j=1}^{N_{\text{bc}}} \left|U_\psi(x^b_j, t^b_j)\right|^2 . \tag{10}$$

Here, $\{(x_j, t_j)\}_{j=1}^{N_{\text{int}}}$ are interior collocation points in Q, $\{(x^b_j, t^b_j)\}_{j=1}^{N_{\text{bc}}}$ are boundary collocation points on Σ, and $\{x_i\}_{i=1}^{N_{\text{ic}}}$ are points in Ω for the initial condition. The weights $w_{\text{res}}, w_{\text{bc}}, w_{\text{ic}} > 0$ balance the different loss components. The tracking term $\mathcal{L}_{\text{track}}$ approximates the first term of the cost functional, while $\mathcal{L}_{\text{reg}}$ approximates the control cost.

This formulation is conceptually simple and does not require the adjoint variable. However, as shown in recent studies [1], the soft penalty on the boundary condition may lead to inaccurate satisfaction of the constraint, and the loss balance can be delicate.

2.2 Indirect PINN Formulation with Hard Constraint

The indirect approach exploits the first-order optimality system derived from the Lagrangian. For the Dirichlet boundary control problem, the optimality system consists of the state equation, the adjoint equation, and the stationarity condition. We now detail the neural network parameterization and the hard-constraint mechanism for the Dirichlet boundary condition.

2.2.1 Optimality System

The Lagrangian for the problem is:

$$\mathcal{L}(y, u, \lambda) = J(y, u) + \int_0^T \int_\Omega \lambda \left(\partial_t y - \nu \Delta y + f(y)\right) dx\, dt, \tag{11}$$

where λ is the adjoint variable. Taking variations with respect to y, u, and λ, we obtain the following system:

$$\partial_t y - \nu \Delta y + f(y) = 0, \qquad \text{in } Q, \tag{12}$$

$$-\partial_t \lambda - \nu \Delta \lambda + f'(y)\lambda = -(y - y_d), \qquad \text{in } Q, \tag{13}$$

$$\alpha u - \partial_n \lambda = 0, \qquad \text{on } \Sigma, \tag{14}$$

$$y = u, \qquad \text{on } \Sigma, \tag{15}$$

$$\lambda = 0, \qquad \text{on } \Sigma, \tag{16}$$

$$y(x, 0) = y_0(x), \qquad \text{in } \Omega, \tag{17}$$

$$\lambda(x, T) = 0, \qquad \text{in } \Omega. \tag{18}$$

The condition (14) is the stationarity condition for the Dirichlet case, linking the normal derivative of λ on the boundary to the control u.

2.2.2 Neural Network Parameterization

We introduce three neural networks:

$$y \approx Y_\theta(x, t), \qquad u \approx U_\psi(x, t), \qquad \lambda \approx \Lambda_\phi(x, t), \tag{19}$$

where θ, ψ, ϕ are the parameters. Instead of treating the boundary condition $y = u$ as a soft penalty, we enforce it *exactly* via a hard-constraint construction.

2.2.3 Hard-Constraint for Dirichlet Boundary Condition

Let $d(x)$ be a smooth function such that:

- $d(x) = 0$ for $x \in \partial\Omega$,
- $d(x) > 0$ for $x \in \Omega$.

For simple domains like a rectangle, $d(x)$ can be chosen as the distance to the boundary, e.g., for $\Omega = [0, 1]$, $d(x) = x(1 - x)$. Then we construct the trial function for the state as:

$$Y_\theta(x, t) = U_\psi(x, t) + d(x) \cdot N_\theta(x, t), \tag{20}$$

where $N_\theta(x, t)$ is an additional neural network (or can be merged into Y_θ by modifying the architecture). On the boundary, $d = 0$, we have $Y_\theta = U_\psi$, so the Dirichlet boundary condition is satisfied exactly, regardless of the network parameters.

We further impose the adjoint boundary condition $\lambda = 0$ on Σ via a similar construction, or we can enforce it softly. Since the adjoint boundary condition is homogeneous Dirichlet, we can define:

$$\Lambda_\phi(x, t) = d(x) \cdot M_\phi(x, t), \tag{21}$$

where M_ϕ is another neural network. This ensures $\Lambda_\phi = 0$ on $\partial\Omega$ exactly.

2.2.4 Indirect PINN Loss

With the hard constraints, the loss function for the indirect method contains only the residual errors of the state equation, the adjoint equation, the stationarity condition, and the initial/terminal conditions. The boundary conditions are already satisfied by construction, so no boundary loss terms are needed. The loss is:

$$\begin{aligned}\mathcal{L}_{\text{ind}}(\theta, \psi, \phi) = w_y \mathcal{L}^y_{\text{res}} + w_\lambda \mathcal{L}^\lambda_{\text{res}} + w_{\text{st}} \mathcal{L}_{\text{st}} \\ + w_{\text{ic}} \mathcal{L}^y_{\text{ic}} + w_T \mathcal{L}^\lambda_T,\end{aligned} \tag{22}$$

where the residual terms are defined as follows:

$$\mathcal{L}^y_{\text{res}} := \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} |\partial_t Y_\theta - \nu \Delta Y_\theta + f(Y_\theta)|^2_{(x_j, t_j)}, \tag{23}$$

$$\mathcal{L}^\lambda_{\text{res}} := \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} |-\partial_t \Lambda_\phi - \nu \Delta \Lambda_\phi + f'(Y_\theta)\Lambda_\phi + (Y_\theta - y_d)|^2_{(x_j, t_j)}, \tag{24}$$

$$\mathcal{L}_{\text{st}} := \frac{1}{N_{\text{bc}}} \sum_{j=1}^{N_{\text{bc}}} \left|\alpha U_\psi(x^b_j, t^b_j) - \partial_n \Lambda_\phi(x^b_j, t^b_j)\right|^2, \tag{25}$$

$$\mathcal{L}^y_{\text{ic}} := \frac{1}{N_{\text{ic}}} \sum_{i=1}^{N_{\text{ic}}} |Y_\theta(x_i, 0) - y_0(x_i)|^2, \tag{26}$$

$$\mathcal{L}^\lambda_T := \frac{1}{N_T} \sum_{i=1}^{N_T} |\Lambda_\phi(x_i, T)|^2. \tag{27}$$

Here, $\partial_n \Lambda_\phi$ is computed via automatic differentiation. The stationarity loss $\mathcal{L}_{\text{st}}$ enforces the condition $\alpha u = \partial_n \lambda$ on the boundary. The terminal condition for the adjoint is $\lambda(x, T) = 0$.

Compared to the direct formulation, the indirect loss is more structured: minimizing it drives the neural networks toward a solution of the optimality system, which is precisely the KKT point of the original problem. In particular, because the boundary conditions are hard-constrained, there is no trade-off between satisfying the PDE and satisfying the boundary conditions; the optimizer focuses solely on the residuals.

3 Error Analysis

In this section, we establish an error estimation framework for the proposed Indirect PINN method with hard constraint. We decompose the total error into several components, derive a

conditional a priori error bound, and construct a computable a posteriori error indicator that is consistent with the meshfree, collocation-based nature of PINNs. Unlike finite-element residual estimators, our indicator does not rely on a computational mesh and therefore contains no inter-element jump terms.

3.1 The Optimality System and Its Residuals

Recall the first-order optimality system for our problem: find (y, u, λ) such that

$$\partial_t y - \nu\Delta y + f(y) = 0, \qquad \text{in } Q, \tag{28}$$
$$-\partial_t \lambda - \nu\Delta\lambda + f'(y)\lambda = -(y - y_d), \qquad \text{in } Q, \tag{29}$$
$$\alpha u - \partial_n \lambda = 0, \qquad \text{on } \Sigma, \tag{30}$$
$$y = u, \qquad \text{on } \Sigma, \tag{31}$$
$$\lambda = 0, \qquad \text{on } \Sigma, \tag{32}$$
$$y(x, 0) = y_0(x), \qquad \text{in } \Omega, \tag{33}$$
$$\lambda(x, T) = 0, \qquad \text{in } \Omega. \tag{34}$$

This is precisely the system (12)–(32) of Section 2, and we use the same constant α as in the cost functional (1) (we no longer introduce the separate symbols β_Q, β_R used in an earlier draft, to keep notation consistent across sections; the tracking term always carries weight 1 and the control term always carries weight α).

Let Y_θ, U_ψ, Λ_ϕ be the neural network approximations of y, u, λ built with the hard-constraint construction of Section 2.2.3, so that the Dirichlet conditions (31)–(32) hold *exactly, for every parameter value*. We define the residuals of the optimality system:

$$\mathcal{R}_y(x, t; \theta, \psi) := \partial_t Y_\theta - \nu\Delta Y_\theta + f(Y_\theta), \tag{35}$$
$$\mathcal{R}_\lambda(x, t; \theta, \phi) := -\partial_t \Lambda_\phi - \nu\Delta\Lambda_\phi + f'(Y_\theta)\Lambda_\phi + (Y_\theta - y_d), \tag{36}$$
$$\mathcal{R}_u(x, t; \psi, \phi) := \alpha U_\psi - \partial_n \Lambda_\phi, \quad (x, t) \in \Sigma. \tag{37}$$

Because the boundary and initial/terminal conditions are enforced by construction (hard constraints) rather than penalized, the only residuals that remain are the PDE residuals (35)–(37) together with the (typically small, if the hard constraint for the initial condition is not also imposed exactly) initial/terminal mismatches. The indirect PINN loss, matching (22), is

$$\mathcal{L}_{\text{ind}}(\theta, \psi, \phi) = w_y\mathcal{L}_y + w_\lambda\mathcal{L}_\lambda + w_u\mathcal{L}_u + w_{\text{ic}}\mathcal{L}_{\text{ic}} + w_T\mathcal{L}_T, \tag{38}$$

where

$$\mathcal{L}_y := \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} |\mathcal{R}_y(x_j, t_j; \theta, \psi)|^2, \tag{39}$$
$$\mathcal{L}_\lambda := \frac{1}{N_{\text{int}}}\sum_{j=1}^{N_{\text{int}}} |\mathcal{R}_\lambda(x_j, t_j; \theta, \phi)|^2, \tag{40}$$
$$\mathcal{L}_u := \frac{1}{N_{\text{bc}}}\sum_{j=1}^{N_{\text{bc}}} |\mathcal{R}_u(x_j^b, t_j^b; \psi, \phi)|^2, \tag{41}$$
$$\mathcal{L}_{\text{ic}} := \frac{1}{N_{\text{ic}}}\sum_{i=1}^{N_{\text{ic}}} |Y_\theta(x_i, 0) - y_0(x_i)|^2, \tag{42}$$
$$\mathcal{L}_T := \frac{1}{N_T}\sum_{i=1}^{N_T} |\Lambda_\phi(x_i, T)|^2. \tag{43}$$

Each $\mathcal{L}_\bullet$ is a Monte-Carlo (or quasi-Monte-Carlo) quadrature approximation of a continuous L^2-norm of the corresponding residual, e.g. $\mathcal{L}_y \approx \|\mathcal{R}_y\|^2_{L^2(Q)}$; this correspondence is what links the *discrete, trainable* loss to the *continuous* PDE residuals used in the estimates below.

3.2 Standing Assumptions

The estimates in this section are conditional on the following standing assumptions, which we state explicitly rather than leaving implicit.

(A1) **Well-posedness.** The optimality system (28)–(32) admits a unique solution (y^*, u^*, λ^*) in an appropriate energy class, and the linearization of this system about (y^*, λ^*) satisfies an inf-sup (stability) condition with constant C_{stab}.

(A2) **Regularity of the Dirichlet control problem.** Dirichlet boundary control of a parabolic equation is, in general, a low-regularity problem: for boundary data $u \in L^2(\Sigma)$ the state y is typically only a *very weak solution* (transposition solution), and the trace $\partial_n \lambda|_\Sigma$ appearing in (30) need not exist in the classical sense (see e.g. Lasiecka & Triggiani; Tröltzsch, *Optimal Control of Partial Differential Equations*, Ch. 5). We assume throughout that the data (y_0, y_d, u^*) are smooth enough, and Ω is regular enough, that (y^*, λ^*) lie in $H^{2,1}(Q)$-type spaces in which all traces below are classically well defined. This is a genuine restriction, not a technicality; results that require only $u \in L^2(\Sigma)$ are outside the scope of the estimates below.

(A3) **Nonlinearity.** $f \in C^1(\mathbb{R})$ and f' is bounded below, $f'(s) \geq -c_f$ for some $c_f \geq 0$ (satisfied by the Allen–Cahn nonlinearity $f(y) = y^3 - y$, for which $f'(s) = 3s^2 - 1 \geq -1$).

(A4) **Approximation capacity.** The network architectures for $Y_\theta, U_\psi, \Lambda_\phi$ are rich enough that the best-approximation error ϵ_{net} defined in (46) can be made arbitrarily small by increasing width/depth. No explicit rate in network size is claimed here; this is treated as an assumption, not a proven property of the architecture.

3.3 Decomposition of the Total Error

Let $e_y := y^* - Y_\theta$, $e_\lambda := \lambda^* - \Lambda_\phi$, $e_u := u^* - U_\psi$ denote the errors in the state, adjoint, and control, relative to the exact solution of the optimality system (not to a discretized/mesh solution). The total error is measured in the norm

$$\|(e_y, e_\lambda, e_u)\|_{\mathcal{E}}^2 := \|e_y\|_{L^2(0,T;H^1(\Omega))}^2 + \|e_\lambda\|_{L^2(0,T;H^1(\Omega))}^2 + \|e_u\|_{L^2(\Sigma)}^2. \tag{44}$$

Using the triangle inequality and the stability assumption (A1), the total error can be decomposed into three main contributions:

$$\|(e_y, e_\lambda, e_u)\|_{\mathcal{E}} \lesssim \mathcal{E}_{\mathrm{approx}} + \mathcal{E}_{\mathrm{opt}} + \mathcal{E}_{\mathrm{quad}}, \tag{45}$$

where:

- $\mathcal{E}_{\mathrm{approx}}$: best-approximation error of the neural network architecture (item (A4)),
- $\mathcal{E}_{\mathrm{opt}}$: optimization error, i.e. how far the trained loss (38) is from its infimum over the chosen architecture,
- $\mathcal{E}_{\mathrm{quad}}$: quadrature (sampling) error between the Monte-Carlo loss $\mathcal{L}_{\mathrm{ind}}$ and the continuous residual norms it estimates.

We no longer list a separate $\mathcal{E}_{\mathrm{bc}}$ term: because the Dirichlet conditions are hard-constrained (Section 2.2.3), the boundary/initial-condition error is identically zero at the level of the construction and does not contribute to (45); it can only re-enter indirectly through $\mathcal{E}_{\mathrm{approx}}$ if the auxiliary networks N_θ, M_ϕ are not expressive enough.

3.4 A Priori Error Estimate

Theorem 1 (Conditional A Priori Error Estimate)**.** *Under Assumptions (A1)–(A3), let (y^*, u^*, λ^*) be the exact solution of (28)–(32). Suppose the networks Y_θ, U_ψ, Λ_ϕ can approximate (y^*, λ^*, u^*) to accuracy ϵ_{net} in the $\mathcal{E}$-norm,*

$$\inf_{\theta,\psi,\phi} \|(y^* - Y_\theta, \lambda^* - \Lambda_\phi, u^* - U_\psi)\|_{\mathcal{E}} \leq \epsilon_{net}, \tag{46}$$

and suppose training reaches parameters $(\theta^*, \psi^*, \phi^*)$ with continuous residual norm

$$\|\mathcal{R}_y\|_{L^2(Q)}^2 + \|\mathcal{R}_\lambda\|_{L^2(Q)}^2 + \|\mathcal{R}_u\|_{L^2(\Sigma)}^2 \;\leq\; \epsilon_{loss}^2. \tag{47}$$

Then

$$\|(y^* - Y_\theta^*, \lambda^* - \Lambda_\phi^*, u^* - U_\psi^*)\|_{\mathcal{E}} \;\lesssim\; \epsilon_{net} + C_{stab}\,\epsilon_{loss}, \tag{48}$$

with C_{stab} the stability constant of Assumption (A1).

Sketch of proof. Write $e = (e_y, e_\lambda, e_u)$ for the error at the trained parameters, and split $e = e^{\text{approx}} + e^{\text{res}}$, where e^{approx} is the error of the *best* network approximation (controlled by ϵ_{net} via (A4)) and $e^{\text{res}} := (y^* - Y_\theta^*, \dots) - e^{\text{approx}}$ is what remains. By linearity, e^{res} solves the linearization of (28)–(30) about (y^*, λ^*) with right-hand side equal to $(\mathcal{R}_y, \mathcal{R}_\lambda, \mathcal{R}_u)$ evaluated at the trained networks (this uses (A2) to make the linearization and its traces meaningful, and (A3) to control the sign of the reaction term $f'(y^*)$ so that the linearized adjoint operator is coercive up to a lower-order term, via a standard exponential-weight / Gårding-inequality argument). The stability assumption (A1) then gives $\|e^{\text{res}}\|_{\mathcal{E}} \le C_{\text{stab}}\big(\|\mathcal{R}_y\|_{L^2(Q)} + \|\mathcal{R}_\lambda\|_{L^2(Q)} + \|\mathcal{R}_u\|_{L^2(\Sigma)}\big) \le C_{\text{stab}}\,\epsilon_{\text{loss}}$. Combining with $\|e^{\text{approx}}\|_{\mathcal{E}} \le \epsilon_{\text{net}}$ and the triangle inequality gives (48). We emphasize that this is a *sketch*: a full proof requires making (A1)–(A2) precise (function spaces, trace theorems) for the specific f and domain considered, which is not carried out here. □

Remark 1. *Bound* (48) *uses the* continuous *residual norm ϵ_{loss}, not the Monte-Carlo loss $\mathcal{L}_{ind}$ that is actually minimized during training. The gap between the two is exactly $\mathcal{E}_{quad}$ in* (45)*; see Section 3.6 for how it is controlled in practice.*

Remark 2. *For the Allen–Cahn nonlinearity $f(y) = y^3 - y$, $f'(s) = 3s^2 - 1 \ge -1$ is bounded below, which is what makes the coercivity argument in the proof sketch available; a genuine coercivity estimate for the linearized adjoint operator additionally requires either a smallness condition on T or an exponential time-weight $e^{-c_f t}$ to absorb the -1 reaction term, which we do not carry out in detail here.*

3.5 A Posteriori Error Estimation

Because $Y_\theta, U_\psi, \Lambda_\phi$ are smooth functions on all of $\overline{Q}$ (for smooth activation functions), there is no computational mesh and no notion of an inter-element jump in the normal derivative — such jumps are identically zero for a globally differentiable network. We therefore build the a posteriori indicator directly from pointwise residual evaluation on a *verification point set*, distinct from (and typically much larger/denser than) the training collocation set, rather than from a finite-element-style mesh.

3.5.1 Verification-Point Residual Indicator

Let $\widehat{Q} = \{(x_k, t_k)\}_{k=1}^K \subset Q$ and $\widehat{\Sigma} = \{(x_k^b, t_k^b)\}_{k=1}^{K_b} \subset \Sigma$ be dense verification point sets (e.g. a fixed low-discrepancy or grid sample, held out from training) with associated quadrature weights $\{\omega_k\}$, $\{\omega_k^b\}$ summing to $|Q|$ and $|\Sigma|$ respectively. Define the global a posteriori indicator

$$\eta^2 \;:=\; \sum_{k=1}^{K} \omega_k\,|\mathcal{R}_y(x_k,t_k)|^2 \;+\; \sum_{k=1}^{K} \omega_k\,|\mathcal{R}_\lambda(x_k,t_k)|^2 \;+\; \sum_{k=1}^{K_b} \omega_k^b\,|\mathcal{R}_u(x_k^b,t_k^b)|^2 \;+\; \eta_{\text{ic}}^2 + \eta_T^2, \tag{49}$$

where η_{ic}^2 and η_T^2 are the analogous quadrature approximations of $\|Y_\theta(\cdot,0) - y_0\|_{L^2(\Omega)}^2$ and $\|\Lambda_\phi(\cdot,T)\|_{L^2(\Omega)}^2$. By construction η^2 is a quadrature estimate of the continuous quantity $\|\mathcal{R}_y\|_{L^2(Q)}^2 + \|\mathcal{R}_\lambda\|_{L^2(Q)}^2 + \|\mathcal{R}_u\|_{L^2(\Sigma)}^2 + \|Y_\theta(\cdot,0) - y_0\|^2 + \|\Lambda_\phi(\cdot,T)\|^2$ that enters Theorem 1, evaluated on a point set independent of the one used for training. If desired, a *local* (space-time cell) version can still be reported for visualization, $\eta^2 = \sum_E \eta^2(E)$ with E ranging over cells of a fixed post-hoc partition of $\overline{Q}$ used only for plotting, but we stress that this partition plays no role in the estimator's derivation and no jump terms are included.

3.5.2 Reliability

Theorem 2 (Reliability)**.** *Under Assumptions (A1)–(A3), and with η as in* (49) *computed on a sufficiently fine verification set,*

$$\|(e_y, e_\lambda, e_u)\|_{\mathcal{E}} \;\lesssim\; \eta + \mathcal{E}_{quad}, \tag{50}$$

where $\mathcal{E}_{quad}$ is the (typically negligible, if K, K_b are large) quadrature error of (49) *itself relative to the exact L^2 residual norms.*

Sketch of proof. Immediate from Theorem 1 once one identifies η as a quadrature approximation of ϵ_{loss}: apply (48) with ϵ_{loss} replaced by $\eta + \mathcal{E}_{\text{quad}}$ and ϵ_{net} absorbed into the implicit constant (which is legitimate once the network has been trained to near-zero loss, so that ϵ_{net} is itself of the same order as η). □

3.5.3 On "Efficiency" (Lower Bound)

A finite-element a posteriori estimator is typically also *efficient*: $\eta(E) \lesssim \|e\|_{1,D_E}$ up to data oscillation, proved with element-wise bubble functions supported on individual mesh cells. This construction relies essentially on having a mesh with compactly supported, element-local test functions. In the present meshfree PINN setting there is no such local test-function machinery available in general, and we are not aware of a proof of a matching lower bound for collocation-based PINN residual indicators of this type. We therefore do *not* claim an efficiency theorem here; (49) should be understood as a reliable (upper-bounding, up to constants) indicator only, useful for monitoring convergence and flagging poorly-resolved regions of Q, but not as a certified two-sided estimator. Establishing efficiency rigorously in the meshfree setting is left as an open direction.

3.6 Loss Function as an Error Indicator

Combining Theorem 1 with the identification of η as a quadrature estimate of ϵ_{loss}, we have informally

$$\|(e_y, e_\lambda, e_u)\|_{\mathcal{E}}^2 \;\lesssim\; \mathcal{L}_{\text{ind}} + \mathcal{E}_{\text{approx}} + \mathcal{E}_{\text{quad}}, \tag{51}$$

so a small training loss is *necessary but not sufficient* for a small true error: it must be accompanied by (i) a sufficiently expressive architecture ($\mathcal{E}_{\text{approx}}$ small) and (ii) a sufficiently dense/well-distributed collocation set ($\mathcal{E}_{\text{quad}}$ small). In particular, a low training loss evaluated *on the training points themselves* can be misleading if the network overfits those points; this is precisely why η in Section 3.5 is evaluated on an independent verification set.

3.7 Practical Implementation of the Error Estimator

The a posteriori indicator is computed after training as follows:

1. Sample a verification point set $\widehat{Q} \cup \widehat{\Sigma}$, independent of the training collocation points (e.g. a fresh random draw, or a fixed low-discrepancy grid), together with quadrature weights.
2. Evaluate $Y_\theta, U_\psi, \Lambda_\phi$ and their required derivatives at these points via automatic differentiation, and compute the residuals $\mathcal{R}_y, \mathcal{R}_\lambda, \mathcal{R}_u$.
3. Form the weighted sum (49).
4. (Optional, diagnostic only) Bin the pointwise squared residuals over a post-hoc space-time partition purely for visualization of where the error is concentrated; this partition does not enter the estimator's value.

The computational cost is comparable to one evaluation of the loss function on $K + K_b$ points, i.e. negligible relative to training.

3.8 Numerical Validation Strategy

To validate the error indicator, we propose the following procedure:

1. Construct a reference solution $(y_{\text{ref}}, u_{\text{ref}}, \lambda_{\text{ref}})$ with a fine-grid FEM/adjoint solver.
2. Train the Indirect PINN with an increasing number of training collocation points $N = N_{\text{int}} = N_{\text{bc}}$ (there is no mesh size h in the meshfree setting; N plays the analogous role).
3. Compute the true error $\|e\|_{\mathcal{E}}$ against the reference solution.
4. Compute the indicator η from (49) on a verification set independent of both the N training points and the reference-solution grid.
5. Compare η with the true error as N increases, and report the empirical ratio $\eta/\|e\|_{\mathcal{E}}$ (an *effectivity index*) rather than assuming a fixed algebraic rate; unlike FEM, PINN convergence in N is not generally guaranteed to follow a clean power law, and any observed rate should be reported empirically rather than asserted a priori.

This validates the reliability of the estimator in the sense of Theorem 2; it does not, and is not intended to, validate an efficiency claim (see Section 3.5.3).

3.9 Summary of the Error Estimation Framework

The proposed error estimation framework provides:

- **A conditional a priori bound** (Theorem 1), valid under explicit well-posedness, regularity, and approximation-capacity assumptions (A1)–(A4).
- **A computable a posteriori indicator** (49), built from pointwise residual evaluation on a verification set, with no artificial mesh-jump terms.
- **A reliability theorem** (upper bound), together with an honest statement that a matching efficiency (lower bound) result is not established here for the meshfree setting.
- **Practical guidelines** for computing and validating the indicator empirically.

Remark 3. *The Dirichlet boundary control setting analyzed here (Assumption (A2)) is a known source of low-regularity difficulties for the classical PDE theory; results here should not be read as removing that difficulty, only as working under regularity hypotheses strong enough to bypass it. Extending the framework to genuinely low-regularity Dirichlet data, and to a provably efficient meshfree a posteriori indicator, are left for future work.*

4 Numerical Experiment Setup

4.1 Test Problem

To verify and compare the two PINN formulations presented in Section 2, we construct a test problem based on the *method of manufactured solutions*, which allows the error to be computed against a known exact solution.

We consider the case $d = 1$, $\Omega = (0, 1)$, $T = 1$, and set the diffusion coefficient to $\nu = 0.1$. For the initial verification, the nonlinear term is chosen as $f(y) = y^3$.

We prescribe a reference state and control (the manufactured exact solution) as follows:

$$y^*(x, t) = \sin(\pi x)\, e^{-t}, \tag{52}$$

$$u^*(x, t) = y^*(x, t)\big|_{x\in\partial\Omega} = 0, \quad \forall t \in (0, T), \tag{53}$$

since $\sin(\pi x) = 0$ at $x = 0$ and $x = 1$. The corresponding initial condition follows directly:

$$y_0(x) = y^*(x, 0) = \sin(\pi x). \tag{54}$$

Since the state equation (2) contains no source term, the function y^* above does not, in general, exactly satisfy the equation with $f(y) = y^3$. We therefore add an artificial source term $g(x, t)$ to the right-hand side of the state equation, obtained by substituting y^* directly into the equation:

$$g(x, t) := \partial_t y^* - \nu\Delta y^* + f(y^*). \tag{55}$$

With this source term, y^* is the exact solution of the augmented state equation. The desired trajectory y_d in the cost functional (1) is chosen to coincide with y^*, i.e., $y_d \equiv y^*$, so that (y^*, u^*) is exactly the optimal solution of the control problem, with optimal cost value $J^* = 0$.

4.2 Neural Network Configuration

Both formulations (direct and indirect) use fully-connected feedforward neural networks with tanh activation, taking (x, t) as input and producing the corresponding scalar output.

Bång 1: Neural network architectures used in the experiments.

Network	Hidden layers	Neurons/layer	Activation
Y_θ (state)	4	50	tanh
U_ψ (control)	3	30	tanh
Λ_ϕ (adjoint, indirect method only)	4	50	tanh

For the indirect formulation, the distance function $d(x)$ used in the hard-constraint construction (**??**) is chosen as

$$d(x) = x(1-x), \tag{56}$$

which satisfies $d(x) = 0$ for $x \in \{0,1\} = \partial\Omega$ and $d(x) > 0$ for $x \in (0,1)$.

Network weights are initialized using Xavier (Glorot uniform) initialization.

4.3 Collocation Points and Sampling Strategy

Training points are sampled using Latin Hypercube Sampling (LHS) over the corresponding domains:

- $N_{\text{int}} = 10\,000$ interior points in $Q = (0,1) \times (0,1)$;
- $N_{\text{bc}} = 2\,000$ boundary points on $\Sigma = \{0,1\} \times (0,1)$;
- $N_{\text{ic}} = 500$ points for the initial condition at $t = 0$;
- $N_T = 500$ points for the terminal condition at $t = T$ (used only in the indirect method, for the condition $\lambda(x,T) = 0$).

These points are resampled every 500 epochs to reduce overfitting to a fixed set of collocation points.

4.4 Training Parameters

Training is carried out in two stages:

1. **Stage 1**: optimization with the Adam algorithm, initial learning rate 10^{-3}, decayed exponentially (decay factor 0.9 every 1 000 epochs), run for 20 000 epochs.
2. **Stage 2**: fine-tuning with the L-BFGS algorithm until convergence or a maximum of 5 000 iterations, to improve accuracy once Adam has brought the solution near a minimum.

Table 2 summarizes the loss weights used for each method.

Bảng 2: Loss term weights for each formulation.

Term	Direct PINN	Indirect PINN
w_{res} (state PDE residual)	1	1
w_{bc} (state boundary condition, soft)	10	— (hard-constrained)
w_{ic} (initial condition)	10	10
w_λ (adjoint PDE residual)	—	1
w_{st} (stationarity condition)	—	1
w_T (adjoint terminal condition)	—	10

The Tikhonov regularization parameter is fixed at $\alpha = 10^{-2}$ throughout all experiments. Each configuration is trained over 5 independent runs with different random seeds to assess the stability of the results.

Tài liệu

[1] Z. Zhang, S. Liu, A. Alla, J. Darbon, and G. E. Karniadakis, "Pinns in pde constrained optimal control problems: Direct vs indirect methods," *arXiv preprint arXiv:2604.04920*, 2026.